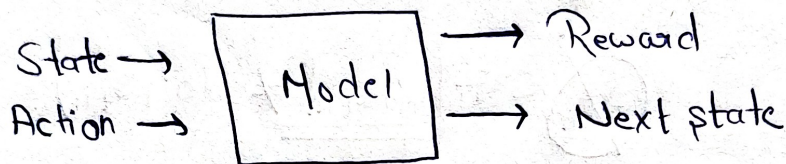


Week 4 →

What is Model? —



Types →

- 1) Sample Model → Produces an outcome drawn from some underlying probabilities.
- 2) Distribution Model → It completely specifies the likelihood or probability of every outcome.

Sample

These procedurally generate samples, without explicitly storing the probability of each outcome.

Distribution

Distribution models contain a list of all outcomes and their probabilities.

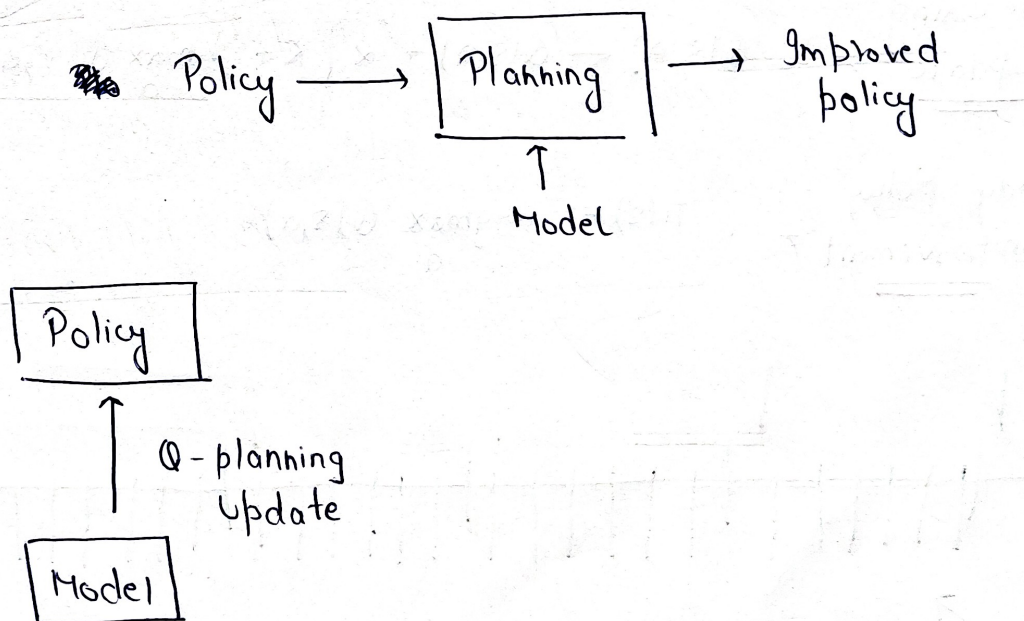
eg. Rolling 12 dice ÷

(35)

- # Rolling a single dice 12 times is an example of sample model. Programitically, one would generate a random numbers from 1-6, 12 times. This is very compact and does not consider joint probabilities.
- # Modelling the complete joint probabilities of rolling 12 dice is much more work. We must consider every possible outcome of 12 dice and assign a probability to each outcome.

Random Tabular Q-planning →

- # Planning is a process which takes a model as input and produces an improved policy.



- # Random Sample one step tabular Q-planning →

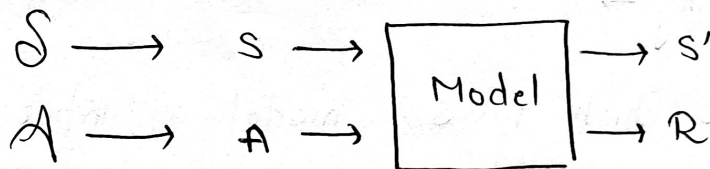
- # This approach assumes we have a sample model of the transition dynamics.

✗ It also assumes that we have a strategy for sampling relevant state action pairs.

✗ One possible option is to sample states and actions uniformly.

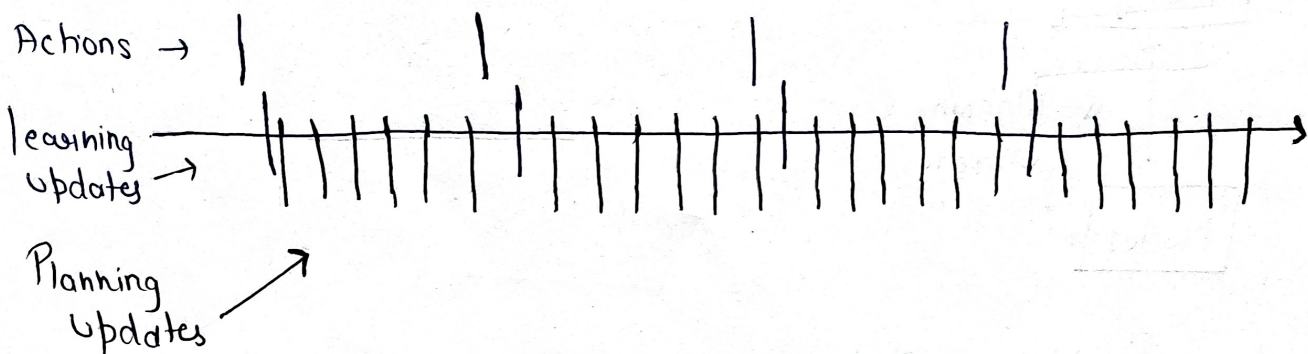
1) Sampling :- This algorithm first chooses the state action pair at random from the set of all states and actions.

✗ It then queries the sample model with the state action pair ~~at random~~ with the state action pair to produce a sample of the next state and reward.



2) Q-learning update :- $Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_a Q(s',a) - Q(s,a)]$

3) Greedy policy improvement :- $\pi(s) = \arg\max_a Q(s,a)$



Planning methods use simulated experience to improve policies.

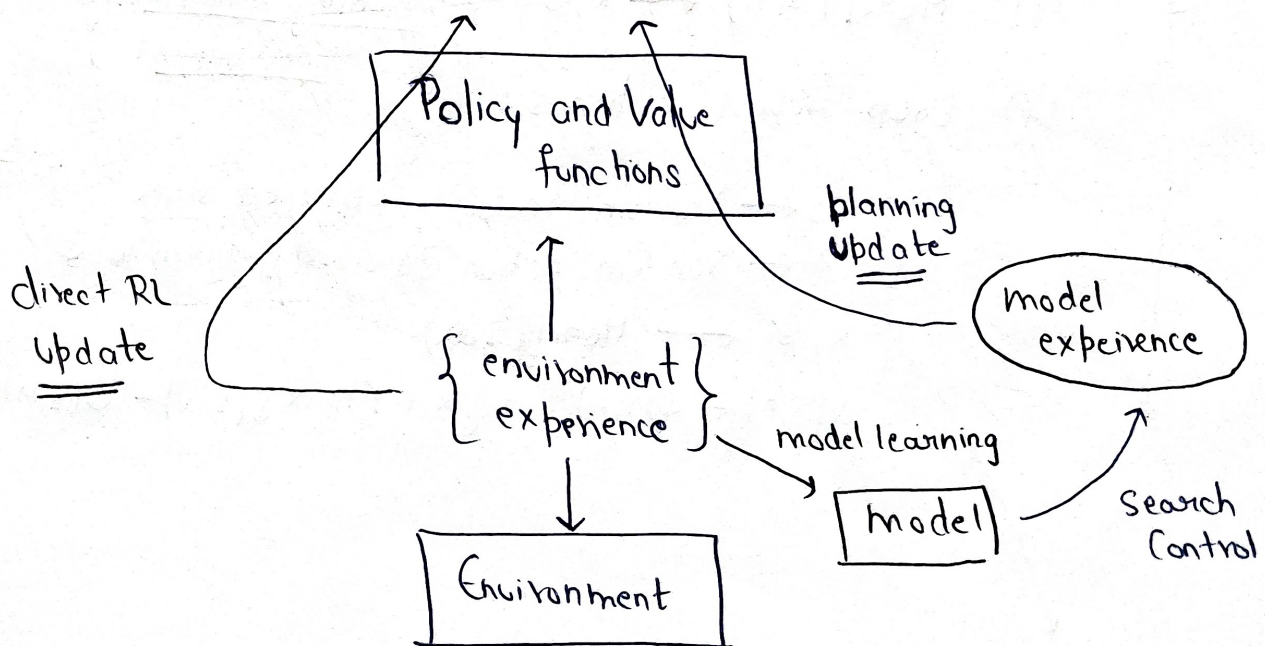
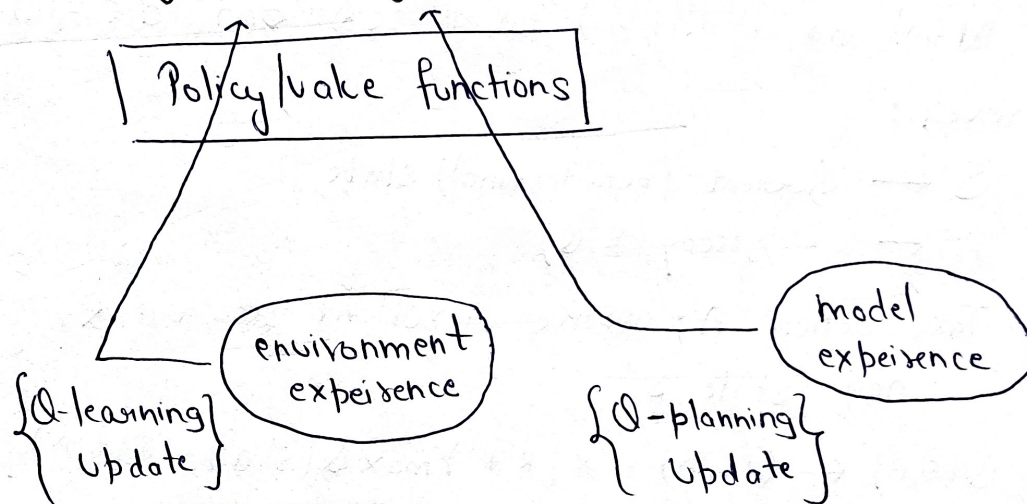
Random-sample one step tabular Q planning uses a sample model to randomly generate experience.

Q-planning's value ~~function~~ function (36)
is then improved using a Q-learning
Updates.

DYNA Architecture $\Rightarrow$

∅ We know two Sources of experience, the environment and the model. In direct RL, the agent learns from experience generated by interacting the world like in Q-learning, but the agent can also generate simulate experience from the model and use it for planning.

Combining Q-learning and Q-planning $\rightarrow$



The environment experience can be used to learn the model. This model will be used to generate model experience. In addition, we want to control how the model generates this simulated experience, what states the agent will plan for, we call this process Search control!

Planning updates are performed using the experience generated by the model.

DYNA algorithm $\rightarrow$ $\left\{ \begin{array}{l} \# \text{ Tabular DYNA-Q assumes} \\ \text{deterministic transitions} \end{array} \right\}$

Tabular DYNA-Q

- 1) Initialize $Q(s,a)$ and $\text{Model}(s,a)$ for all $s \in S$ and $a \in A(s)$
- 2) Loop forever:
 - (a) $S \leftarrow$ current (non-terminal) state
 - (b) $A \leftarrow \epsilon\text{-greedy}(S, Q)$
 - (c) Take action A ; observe resultant reward, R , and state, S' .
 - (d) $Q(s,A) \leftarrow Q(s,A) + \alpha [R + \gamma \max_a Q(s',a) - Q(s,A)]$
 - (e) $\text{Model}(s,A) \leftarrow R, S'$ (assuming deterministic environment)
 - (f) Loop repeat n times:
$$\left\{ \begin{array}{l} S \leftarrow \text{random previously observed state} \\ A \leftarrow \text{random action previously taken in } S \\ R, S' \leftarrow \text{Model}(s,A) \\ Q(s,A) \leftarrow Q(s,A) + \alpha [R + \gamma \max_a Q(s',a) - Q(s,A)] \end{array} \right\}$$

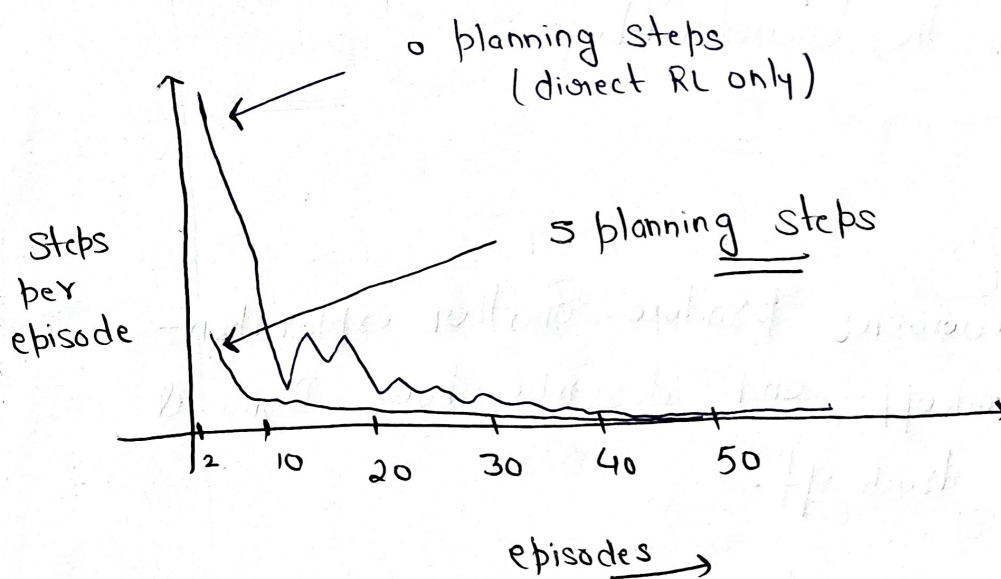
Dyna-Q then perform several steps of planning, each planning

Steps consist of three steps i.e.

(37)

Search Control, model query and value update.

- # In this algorithm, Search Control selects a previously visited state action pair at random. It must be a state action pair the agent has seen before. Otherwise, the model would not know what happens next.
- # Given the state action pair, we create the model for the next state and reward. We have now generated a model transition.
- # Finally, Dyna-Q performs a Q-learning update with the simulated transition.
- # DYNQ-Q performs many planning updates for each environment transition!



{ More planning $\rightarrow$ faster learning }

- # The planning update will not have any effect if the sample state action pairs produces 0 T-error.

How models can be inaccurate?

Models are inaccurate when transitions they store are different from happen in the environment.

The effect of planning with inaccurate model depends on how the model is inaccurate. In the beginning, the model is incomplete. Since the model can't produce a next step or reward, it can't be used for planning.

planning improves the policy with respect to what model thinks will happen, rather than what will really happen in the environment. The model become outdated when the environment changes. Planning with that model will likely make the agent's policy worse with respect to the environment.

Dyna-Q →

∅ How model inaccuracies produce another exploration-exploitation tradeoff and describe how Dyna-Q addresses this trade off.

∅ Agent has to make a choice; explore to make sure it's model is accurate, or exploit the model to compute the optimal policy, assuming that the model is correct!

(37.1)

∅ To encourage, the agent to revisit its state periodically, we can add a bonus to the reward used in planning.

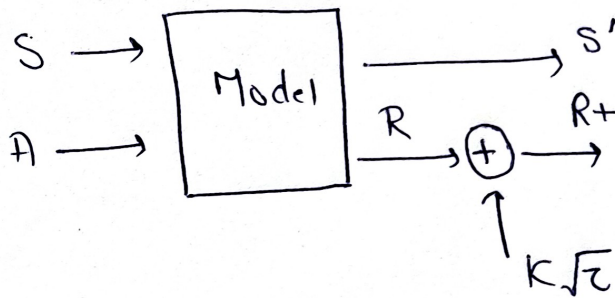
→ Bonus rewards for exploration!

$$\text{New reward} = r_t + k\sqrt{t}$$

actual
reward
from the model.

amount of time it's been
since the state action
pair was last visited
in the environment!

constant to control
the influence of the
bonus of the planning
update



∅ By artificially increasing the rewards used in planning, we increase the value of state action pairs it haven't been visited ~~recently~~ in a long time.